Northeastern University
College of Engineering
Department of Electrical & Computer Engineering

EECE2310 – Introduction to Digital Design and Computer Architecture

Homework 4

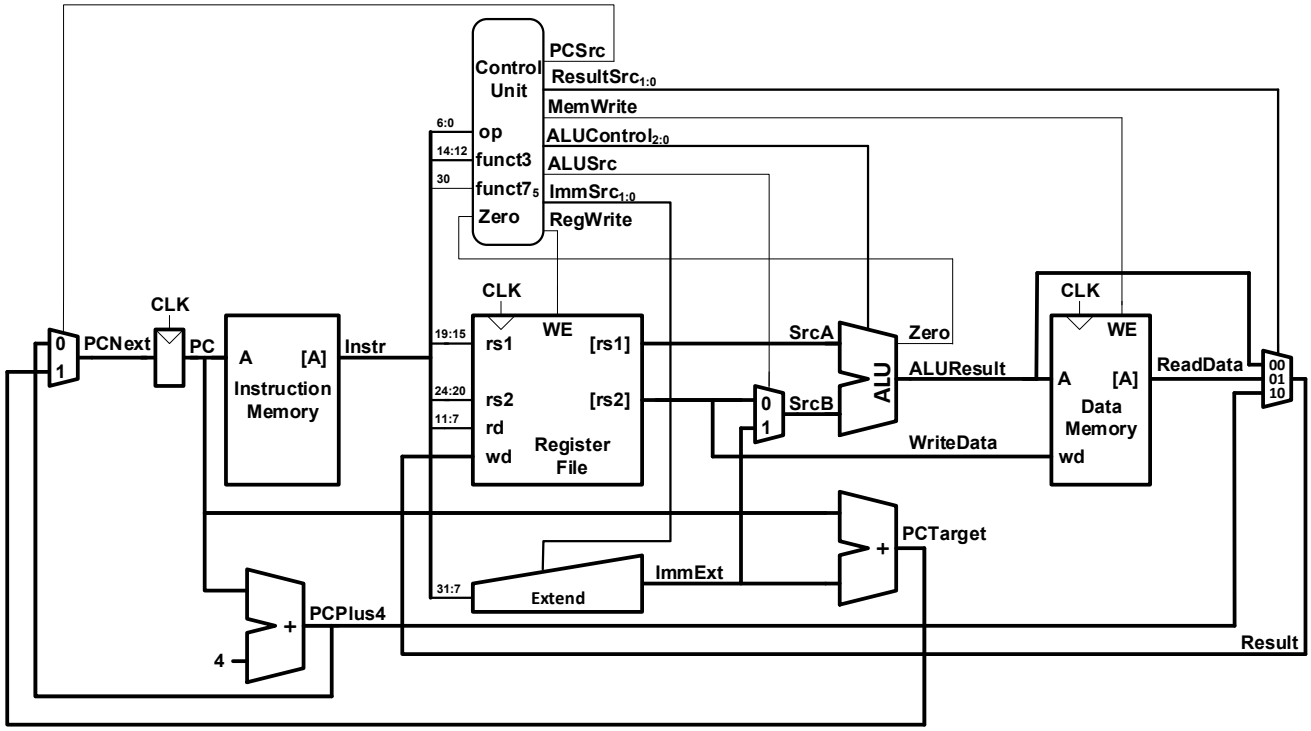
Submission Instructions

- Homework assignments must be done by each student individually (no groups are allowed).
- For the programming problems (RISC-V assembly):
    1. Your code must be well commented by explaining what the lines of your program do. Have at least one comment for every 4 lines of code.
    2. At the beginning of your source code files (only if a separate source code file is required by the question) write your full name, and NU student ID.
    3. If required by the question, test your code using the RARS assembler.
- For non-programming problems, include explanation of all steps of your answers/calculations not only the final results.
- Submit the following to the homework assignment page on Canvas:
    1. Your homework report developed by a word processor (e.g., Microsoft Word) and submitted as <u>one PDF file</u>. Any handwritten answers will not be accepted. For answers that require drawing of diagrams, one option is to use Lucid (check the *Creating Diagrams with Lucid* document on the Miscellaneous Resources page on Canvas). The report includes the following (depending on the assignment contents):
        a. Answers to the non-programming problems that show all the details of the steps you follow to reach these answers.
        b. A summary of your approach to solving the programming problems.
        c. The screen shots of the sample runs of your programs.
    2. If required by any of the questions, your well-commented programs <u>source code files</u> (i.e., the .asm files).

Do NOT submit any files (e.g., the PDF report file and the programs source code files) as a compressed (zipped) package. Rather, upload each file individually.
Do NOT post solutions to any assignment or projects on public repositories like GitHub. Sharing your work in this way can compromise the learning experience for other students and could be considered academic dishonesty.

<u>Note</u>: You can submit multiple attempts for this homework, however, only what you submit in the last attempt will be graded (i.e., all required reports and files must be included in this last attempt).

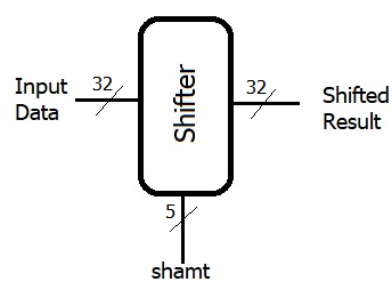
Q1 (25 Points)



Extend the above single-cycle datapath and control block diagram by adding the shifter shown below to implement the **`slli`** instruction. This shifter must be added to the datapath as a separate circuit from the ALU. You also must add a new control signal named **`Shift`**, as we do not want to change the number of inputs to the result source mux. The **`slli`** instruction has the following format (which is similar to the I-type format) **`slli rd,rs1,imm`**

31:20	19:15	14:12	11:7	6:0
[31:25]=0 [24:20] = imm	rs1	0x1	rd	0010011
12 bits	5 bits	3 bits	5 bits	7 bits

The instruction shifts the value in the **`rs1`** left number of bits equal to the 5-bit value stored in the **`imm`** field and stores that value in register **`rd`**. The figure shows the shifter to be used.



a) Draw the extended block diagram to include the new proposed control signal, the shifter circuit, and any additional circuits needed. Make sure to label any added line following the same labeling technique shown in the above block diagram.

b) Extend the *"Control Unit - Instruction Decoder"* table in the lecture slides to reflect the support for the new **`slli`** instruction. Add to the table a new row for the **`slli`** instruction and a new column for the new **`Shift`** signal.

Q2 (25 Points)

Assume the elements in the RISC-V single-cycle datapath and control have the following delays in picoseconds (ignore the delay time through the Control Unit).

Element	Parameter	Delay (ps)
PC clock-to-Q	t_{pcq_PC}	30
Multiplexer	t_{mux}	25
ALU	t_{ALU}	200
Data and Instruction Memory (read or write)	t_{mem}	250
Register file (read or write)	t_{RF}	150
Plus 4 Adder	t_{add4}	70
Extend unit	t_{ext}	15

a) For the following RISC-V add instruction, calculate the total time in *ps* needed from the moment the clock edge triggers the PC to read the instruction address until the sum data is ready to be written to the register file right before the next clock edge.

```
add t1, t2, t3
```

b) Repeat part (a) for the following store instruction. Here you need to calculate the total time in *ps* needed from the moment the clock edge triggers the PC to read the instruction address until the destination address, of the data to be stored, is ready to be provided to the data memory right before the next clock edge.

```
sw t0, 32(s3)
```

Show all the steps of your calculations. Note: This is not the T_c as in the lecture and hence no need to consider the setup time of the register file or the data memory.

Q3. (25 Points)

For a system that uses the **little-endian** data byte ordering and right after executing the following RISC-V instructions, what will be the 32-bit contents of registers **`t1`**, **`t2`**, **`t3`**, **`t4`**, **`t5`** (in **hexadecimal presented as 0xAABBCCDDEE**)? You must show all the steps you used to get your answer and explain your results on a layout that shows the data memory contents for every addressable byte in the memory. Start the data segment at address zero and apply **automatic alignment** on its contents.

```
.data
    AH: .half 0x1234, 0x8899, 0x5555
    AW: .word 0x66667777, 0x00ABCDEF
.text
 main:
    la   t0, AH
    lw   t1, 0(t0)
    lb   t2, 1(t0)
    lb   t3, 2(t0)
    lbu  t4, 3(t0)
    lw   t5, 8(t0)
     ...
```

Q4 (25 Points)

Write a RISC-V program that asks the user to input string. The program then removes all space characters from the string. The program finally displays the resulting string without spaces. The following *C* code shows the proposed algorithm for the program. The string is traversed with two indices, called **`old_index`** and **`new_index`**, where the latter always takes a value less or equal to the former. When a non-space character is found, the character at position **`old_index`** is copied to position **`new_index`**, and both indices are incremented. When a space is found, the current character is not copied, and only **`old_index`** is incremented. The algorithm stops when a *null* character is found.

```c
#include <stdio.h>
int main() {
  // Read string
   char s[100];
   printf("Enter string: ");
   gets(s);
 // Remove spaces
    char c;
    int old_index = 0;
    int new_index = 0;
    do {
    // Read character
     c = s[old_index];
    // Old position moves ahead
     old_index++;
    // If it's a space, ignore
     if (c == ' ') continue;
    // Copy character
     s[new_index] = c;
    // New position moves ahead
     new_index++;
    } while (c);

  // Print result
    printf("New string: %s\n", s);
}
```

a) Write the full well-commented RISC-V program and submit it in a file named **`nospace.asm`**
b) Run your program on **RARS** and include in your homework report screen shots of the program sample runs. Make sure to enter a string containing spaces to demonstrate the correct functionality of the algorithm.

(you can use any of the RARS pseudo instructions in your program)